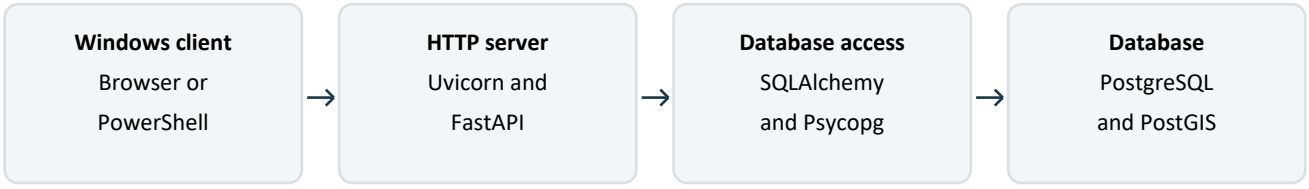
MyStreetVendor Foundation Handbook

Understand the system built in Phases 0 and 1

For a reader who knows Java or C++ and is learning full-stack development.



Windows HTTP: 127.0.0.1:8000

Container database: db:5432

Alembic applies explicit database revisions. A named volume preserves database files.

Read the code. Follow a request. Inspect the database. Explain the evidence.

21 October 2026
Application source checkpoint 699a820
Learning edition with practical exercises and an answer key

Reading this book

Keep the project open beside you. Read in order, predict what the code will do, then compare your prediction with the source or an exercise. This book explains the existing foundation; business features remain future work.

Use four reading sessions: Chapters 1 to 3 for the mental model, 4 to 7 for the application code, 8 to 11 for infrastructure and debugging, and 12 to 14 for the file reference and readiness practice.

Commands use Windows PowerShell. Complete the Chapter 11 session setup before running its command examples. Use the Markdown companion for copying code and recording your own notes. Previously executed results are dated; exercise results are labelled as expected behavior.

Contents

1 What we have built 3

2 Following one request through the system 5

3 Reading the Python code 7

4 Configuration and secrets 8

5 Application startup 9

6 Database connections and sessions 11

7 The health endpoint 12

8 Docker and the local network 14

9 Schema changes with Alembic 16

10 Tests and evidence 18

11 Operating and debugging the foundation 19

12 Every file in the repository 22

13 Practice and readiness for Phase 2 24

14 Glossary and answer key 25

1 What we have built

MyStreetVendor is a planned service connecting nearby customers with street vendors. Eventually a customer will discover a vendor, request a visit and receive updates. The vendor will use a separate application to share availability, receive requests and manage visits. Both applications will use one backend.

We have completed the architecture documentation and the first backend foundation. This book explains that foundation as it actually exists in the repository. Its purpose is to help you trace the code, operate the local services and investigate a failure yourself before approving authentication work.

You already know basic programming in Java or C++. That is enough to begin. You do not need to memorize every Python statement, Docker option or dependency version. Aim to explain what each component owns, which file controls it and what observation would prove it is working.

The boundary of this edition

The source snapshot for this edition is Git commit `699a820`. The earlier architecture checkpoint is `9fda390`. The recorded verification was performed on 21 October 2026. This book was prepared from the committed code and that verification record. Its exercises are instructions for you to run; their expected results are not additional tests that were executed while writing this book.

The repository contains a small FastAPI application, a PostgreSQL/PostGIS container, Docker Compose configuration, validated configuration, a SQLAlchemy connection pool and session dependency, Alembic migrations, ten automated tests and design documents. The only application endpoint is `GET /health`. FastAPI also supplies the documentation routes `/docs`, `/redoc` and `/openapi.json`.

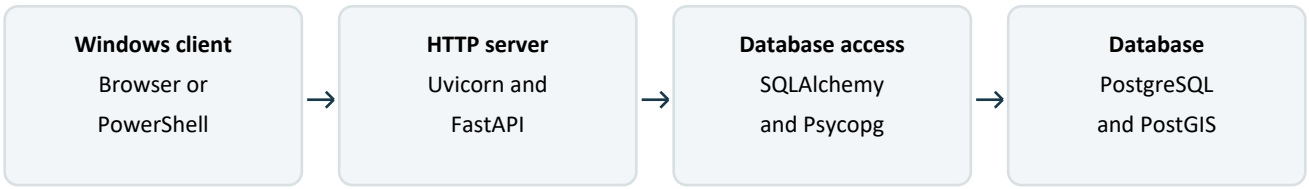
The completed design documents describe the future database and APIs in detail. A documented table or endpoint does not become a running feature merely because its design exists. There are no customer accounts, vendor accounts, login tokens, vendor profiles, GPS endpoints, visit requests, WebSockets, push notifications or mobile applications in the implemented foundation.

Item	State in this edition
System architecture and future contracts	Documented in Phase 0
FastAPI and Uvicorn application	Implemented in Phase 1
PostgreSQL with working PostGIS	Implemented and checked locally
SQLAlchemy and Alembic foundation	Implemented and checked locally
Health responses and basic automated tests	Implemented and checked locally
Authentication and business tables	Planned for their feature phases
Customer and Partner mobile apps	Planned as independent later projects
Public VPS deployment and backups	Designed, not deployed or rehearsed

The ten cases passed on Windows and inside the Linux API image during the recorded verification. They are the same suite run in two environments. A separate default run passed seven unit tests and deliberately skipped the three opt-in integration cases. A live check also observed HTTP 200, then 503 while PostgreSQL

was stopped, then 200 after it restarted without restarting the API. These observations prove the scope described in the verification record; they do not establish production readiness.

The current architecture



Windows HTTP: 127.0.0.1:8000

Container database: db:5432

Alembic applies explicit database revisions. A named volume preserves database files.

Uvicorn receives network traffic. FastAPI chooses the handler and prepares its dependencies. SQLAlchemy provides a session and manages connections. Psycopg sends PostgreSQL protocol messages. PostgreSQL executes the statements, including functions supplied by PostGIS. Docker supplies the environment and network in which the containers run.

Alembic is a separate tool using the same database settings. You run it explicitly to apply a database revision. It is not an extra always-running container and the API does not invoke migrations automatically on startup.

Why this foundation matters

Suppose we added registration before establishing a dependable connection and migration process. A failed request could be caused by password code, an unavailable database, a wrong hostname or a missing table. It would be difficult to isolate the problem. The foundation gives us a small, observable path before business rules arrive.

The architecture is a modular monolith. That means one backend application will grow clear internal feature modules. It does not mean every planned module exists today. It also does not require each module to have its own server. Authentication can be added next while continuing to use the same shared configuration, session management and migration history.

The two mobile applications will remain independent projects. Sharing a trusted backend does not require them to share screens, permissions or release processes. Their future clients will never receive database credentials or connect directly to PostgreSQL.

A realistic learning goal

By the end of this book, you should be able to start the existing stack, explain a health response, identify the current migration, run both kinds of tests and recover from a database outage. You should be able to locate the relevant file before making a change. You should also be comfortable saying that something is only designed, or that a particular test does not prove a broader claim.

Learning check: name one implemented component, one future feature and one piece of evidence supporting the implemented component. A good answer might be the health handler, future login, and the recorded real HTTP response together with its integration test.

2 Following one request through the system

Start with an action you can observe. You enter `http://127.0.0.1:8000/health` in a browser or send an HTTP request from PowerShell. The request asks the server whether its database dependencies are available. It does not ask for a vendor or create a user.

Read the address

The `http` scheme describes the protocol used for this local request. The host `127.0.0.1` means the same computer on which the client runs. Port `8000` selects the API listener exposed on Windows. The path `/health` selects the route. `GET` is the method, meaning this operation reads status rather than creating application data.

When you later use a physical phone, its `127.0.0.1` means the phone, not your development PC. That is why phone access needs deliberate LAN configuration. It has not been set up in this milestone. Likewise, inside the API container, localhost refers to that container. Chapter 8 explains why the application uses the database service name `db` there.

Step by step

Docker forwards the request from the published Windows port to port 8000 in the API container. Uvicorn receives the HTTP message and invokes the FastAPI application. FastAPI finds the `/health` route registered by the health router.

The handler declares that it needs a SQLAlchemy session through `Depends(get_db)`. FastAPI runs that dependency. It creates a session from the factory stored on the application's state and yields it to the handler. There is no global mutable session shared by all users.

The handler executes `SELECT 1`. This small query makes the application use its actual connection configuration and driver. A SQLAlchemy engine is lazy: constructing it earlier did not establish that PostgreSQL could be reached. The query borrows a connection from the pool, opening one if necessary, and Psycopg communicates with PostgreSQL.

If the first query succeeds, the handler executes `SELECT PostGIS_Full_Version()`. A PostgreSQL server can be available while the target database lacks the PostGIS extension. The second query checks that the spatial functionality is usable in this database.

On success, FastAPI serializes a `HealthResponse` into JSON and sends HTTP 200. After the request, the dependency's context manager closes the session. The connection can return to the pool for another request. An implicit read transaction is cleaned up; the handler does not commit business changes.

```
{
  "status": "ok",
  "database": "ok",
  "postgis": "ok"
}
```

If PostgreSQL cannot be reached, the handler returns HTTP 503 with `database` set to `unavailable` and `postgis` set to `unknown`. It does not claim to have checked PostGIS after the first query failed. If the first query succeeds but the spatial query fails, the response reports the successful database check and unavailable PostGIS stage. Chapter 7 explains the exact branches and their limits.

HTTP status and JSON are different parts

HTTP 200 is a protocol status code. The JSON body is application data describing individual checks. A monitoring tool can use the HTTP status without understanding every JSON field. You should inspect both when debugging. A browser displaying JSON alone can hide the status unless you look at its network tools or use an HTTP client that prints headers.

A 404 means a server answered but the requested route was not found. A refused connection means the client did not reach a listener at that address and port. A 503 from this health handler means it reached the application and a dependency check failed. Those are different failure boundaries and call for different investigations.

Connections with what you already know

In Java, you may have seen an object that exposes methods to other code. An API exposes operations across a network boundary. The caller sends a structured request instead of invoking a local method directly. Network communication adds addresses, serialization, timeouts and the possibility that the other process is unavailable.

A database driver plays a role similar to a JDBC driver: it understands the database protocol. SQLAlchemy provides a higher-level toolkit and object/session machinery around that driver. FastAPI and Uvicorn together occupy different responsibilities in the web-serving path; PostgreSQL is a separate program with its own state and lifecycle.

Python context managers provide structured cleanup, similar in purpose to Java try-with-resources or C++ RAII. The implementations differ, but the shared goal is to release a resource even when an operation fails. You will see `with`, `yield`, `try` and `finally` in the session, lifespan and test fixtures because those resources need predictable lifetimes.

Follow the files in reading order

Open `backend/app/main.py` to see how the application is constructed. Then read `config.py` for settings, `database.py` for the engine and session dependency, and `health.py` for the actual request behavior. Keep the request path in mind as you move between them. Chapter 3 introduces the Python syntax before Chapters 4 through 7 explain each responsibility in detail.

After the application code, read `infrastructure/compose.yaml` and `backend/Dockerfile` to see how the processes are supplied and connected. Then read the migration files and tests. This order moves from the action you observe to the environment and evidence that support it.

Learning check: if the database is stopped and you receive the documented 503 response, which parts of the path have already worked? Your client reached the published API listener, Uvicorn and FastAPI routed the request, and the handler detected a failed database operation. The response helps you focus on the remaining boundary.

3 Reading the Python code

The backend is small enough to understand by following one request. You do not need to learn all of Python before reading it. Start with the syntax used in these files, then connect each statement to a responsibility you already recognize from Java or C++.

Python uses indentation to define a block. A function begins with `def`; a class begins with `class`. The indented statements belong to that function or class. Imports make names from another module available. For example, `from app.config import Settings` means that this file uses the `Settings` class defined in the application's configuration module. A Python file is a module, and the `app` directory is an importable package.

Read this real function signature from `backend/app/database.py`:

```
def create_db_engine(settings: Settings) -> Engine:
```

`settings` is the parameter, `Settings` is its type annotation, and `Engine` is the annotated return type. This resembles a Java method taking a configuration object and returning a database-related object. Python annotations help readers, editors and frameworks understand the intended types. They do not make ordinary Python execution enforce every type the way a Java compiler does. Pydantic supplies actual runtime validation for the settings and response models in this project.

In `main.py`, the factory accepts an optional argument:

```
def create_app(settings: Settings | None = None) -> FastAPI:
    if settings is None:
        settings = Settings()
```

The `|` means either type is allowed by the annotation. `None` is Python's absence value, similar in purpose to `null`. The default `= None` allows the caller to omit the argument. Normal server startup omits it; tests can supply known settings. The `is None` check tests that specific absence value. This small choice lets tests construct an application without depending on the developer's private environment file.

You will also see decorators: lines starting with `@`. A decorator registers or changes how the following function is used. The route decorator in `health.py` registers a function as the handler for an HTTP path. `@property` in `config.py` makes a calculation look like an attribute. Thus `settings.database_url` calls the property's function; it is not a second password value stored independently.

The `with` statement manages a resource's lifetime. Think of Java's try-with-resources or a C++ object whose destructor releases a resource. A context manager enters a block, allows work, and performs cleanup when the block exits, including when an exception occurs. The session dependency uses this mechanism to avoid leaving database sessions open.

The related `yield` statement pauses a function and provides a value to its caller. In the database dependency, FastAPI receives the yielded session, runs the endpoint, and then allows the dependency to finish its cleanup. In the lifespan function, `yield` separates the application's running period from shutdown cleanup. These uses are deliberate resource-management boundaries.

The project uses normal `def` for its synchronous database functions and health handler. It uses `async def` for the application's lifespan callback because that callback participates in the asynchronous server lifecycle. This does not turn the PostgreSQL driver into an asynchronous driver. Synchronous FastAPI handlers run through its worker-thread machinery; the database operations here remain synchronous.

Finally, `pyproject.toml` describes the Python package rather than application behavior. It declares Python 3.13, the direct dependency pins, the optional `test` dependency group and Pytest configuration. The

installed versions are the project's recorded selection, not a claim about whatever versions might be newest when you read this book.

Check your understanding

Why can a test call `create_app(settings)` while Uvicorn calls `create_app()`? The argument is optional: a test supplies a controlled object, while normal startup creates settings from configuration sources.

Does `-> Engine` create an engine? No. It describes the intended return type. The function body performs the construction.

Why is `with` important even if a request fails? It gives cleanup a defined exit path when an exception interrupts the normal flow.

4 Configuration and secrets

The application needs to know which database to contact and which credentials to use. Those values belong to configuration because a Windows process, a Docker container and a future production server need different connection details while using the same application code.

`backend/app/config.py` defines the configuration contract. Its central class inherits from Pydantic Settings:

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env", extra="ignore", hide_input_in_errors=True
    )
```

When `Settings()` is constructed, Pydantic Settings reads its configured sources and validates the fields. For the sources used here, explicit constructor values take priority over process environment variables, which take priority over values in the `.env` file, followed by field defaults. This is why test values can override a developer's shell configuration and why Compose can override a host's database hostname.

The path `.env` is relative to the process's current working directory. The Windows instructions therefore start from the repository root. It is not automatically resolved relative to `config.py`. A process launched from another folder might miss the file. Inside the API container, Compose passes the required environment variables explicitly; the root secret file is not copied into the image.

`extra="ignore"` lets the root `.env` also contain Compose-only values such as `API_PORT`. The settings class ignores keys it does not declare. `hide_input_in_errors=True` limits input values in normal validation error strings. These measures reduce accidental disclosure, but they do not make arbitrary application logging safe.

The fields specify both defaults and validation rules:

```
postgres_password: SecretStr = Field(min_length=1)
postgres_host: str = Field(default="127.0.0.1", min_length=1)
postgres_port: int = Field(default=5432, ge=1, le=65535)
db_connect_timeout_seconds: int = Field(default=3, ge=1, le=30)
```

The password has no default, so it must be provided. An empty password is rejected. The host has a local default, the port must be within a valid numeric range, and the connection timeout must be between one and thirty seconds. Environment values arrive as text; Pydantic converts and validates them. A port containing letters should fail during settings construction instead of becoming a confusing later connection error.

`SecretStr` masks a password in normal object representations. It does not encrypt the password on disk or prevent code from revealing it deliberately. The application calls `get_secret_value()` when the driver genuinely needs the credential. Never print that result or export the full connection string to a public issue.

The `database_url` property calls SQLAlchemy's `URL.create` with separate username, password, host, port and database arguments. Building a URL by joining strings can misinterpret password characters such as `@`, `:`, `/` or `%` as URL punctuation. Supplying the components separately preserves the actual password. The configuration tests check this behavior with a fictional password containing special characters.

Distinguish the two environment mechanisms. Compose uses `--env-file .env` to substitute values into its YAML. It then passes the listed `environment` entries into containers. Host Python uses Pydantic to read `.env`. A file being available to Compose does not automatically place every key into every container.

A realistic mistake is setting `POSTGRES_HOST=db` in a Windows terminal. Docker understands that service name on its project network, but Windows Python normally does not. Another mistake is editing the initialization password after a database volume already exists. That does not change the password stored inside PostgreSQL. Investigate which side has the old value before changing credentials through an authorized database operation.

Check your understanding

Why does the same code connect to different hosts in Windows and Docker? Configuration supplies the address appropriate to the process's network.

Is `SecretStr` encrypted storage? No. It is a value wrapper that masks ordinary representations; the underlying secret still needs private storage and careful handling.

Why can the root working directory matter? The configured `.env` path is relative to that directory, so launching elsewhere can change which file is found.

5 Application startup

`backend/app/main.py` is the assembly point. It connects already-defined components rather than containing the database logic itself. The application factory creates configuration if none was supplied, constructs the engine, defines shutdown cleanup, creates FastAPI, attaches shared objects, registers routes and returns the application.

The final assembly is short:

```
app = FastAPI(title="MyStreetVendor API", version="0.1.0", lifespan=lifespan)
app.state.engine = engine
app.state.session_factory = sessionmaker(bind=engine, autoflush=False)
app.include_router(router)
return app
```

`FastAPI(...)` constructs the web application and its generated API metadata. `app.state` is a place to keep application-level objects. Here it holds the engine and a session factory. The session factory is a callable that creates a fresh session when a request needs one; it is not one shared mutable session for all requests. `include_router` installs the health router defined in `health.py`.

Uvicorn runs this application. In the container's command, `app.main:create_app` identifies the Python module and callable. The `--factory` flag says to call that function to obtain an application. `--host 0.0.0.0` tells Uvicorn to listen on the container's network interfaces. Compose separately limits the Windows-published HTTP port to loopback. Those settings refer to different sides of the network boundary.

There is no global `app = create_app()` statement at module level. Importing the module defines its functions without constructing settings or an engine. Tests and tools can import it without triggering configuration-dependent startup. Even constructing the engine is not evidence a database connection succeeded: connections are opened lazily when SQL is first required.

The lifespan callback defines cleanup around the server's running period:

```
@asynccontextmanager
async def lifespan(app: FastAPI) -> AsyncIterator[None]:
    try:
        yield
    finally:
        engine.dispose()
```

Code before `yield` is the startup portion. In this implementation there is no database query in that portion. After the application finishes, the `finally` block disposes the engine's pool. A pool is a managed collection of reusable connections. Disposing it releases the pooled resources as the application shuts down; it is not a database deletion operation.

This explains an important observation: the HTTP server can start even when PostgreSQL is unavailable. Startup constructs the application, while `/health` probes its dependencies. The response can therefore be HTTP 503 even though Uvicorn is listening correctly. Docker's API health check uses the HTTP endpoint to assess dependency readiness.

`autoflush=False` prevents the session from automatically flushing pending ORM changes before certain queries. There are no business model writes in Phase 1, so this has little visible effect yet. It does not mean transactions commit automatically. Future write services will need explicit transaction ownership and commits.

When debugging startup, first separate construction from connectivity. A missing required password can fail settings construction before the listener is ready. A wrong database host can allow construction but cause `/health` to return 503. A missing route registration can leave the server responding with 404 at the expected path. These symptoms point to different responsibilities and should not all lead to rewriting `main.py`.

Check your understanding

What does `--factory` tell Uvicorn? Call `create_app` to obtain the FastAPI application rather than treating the function itself as the application object.

Why can the server listen while health is 503? Opening an HTTP listener and successfully reaching PostgreSQL are separate operations.

Does `engine.dispose()` delete PostgreSQL data? No. It releases resources belonging to the application's connection pool.

6 Database connections and sessions

`backend/app/database.py` separates three concepts: engine, session factory and session. Keeping them distinct makes connection failures easier to understand.

The engine is a long-lived manager for a database URL, driver and connection pool. The session factory, constructed in `main.py`, creates sessions bound to that engine. A session is a request's working interface for SQL and, in future phases, ORM operations. A session can borrow a connection when needed and release it when its work ends. It is not a separate PostgreSQL server and does not permanently own one connection for its entire existence.

The engine's settings make resource limits visible:

```
return create_engine(
    settings.database_url,
    pool_pre_ping=True,
    pool_size=5,
    max_overflow=5,
    pool_timeout=3,
    hide_parameters=True,
    connect_args={
        "connect_timeout": settings.db_connect_timeout_seconds,
        "options": "-c statement_timeout=5000",
    },
)
```

`pool_size=5` configures up to five retained connections in this engine's ordinary pool. They are created as needed, not all opened immediately. `max_overflow=5` permits up to five additional connections during demand; overflow connections are not retained as ordinary pooled capacity. Thus this one engine can reach ten checked-out connections under its configured limits. These numbers apply per engine, not automatically to every process on a server.

There are three different waits. `pool_timeout=3` limits waiting for a pool slot. `connect_timeout`, normally three seconds, bounds an individual driver connection attempt. The PostgreSQL `statement_timeout=5000` option limits SQL statement execution to five thousand milliseconds. These values are not one combined HTTP deadline. A request can pass through several stages, and a database timeout is not permission to repeat an interrupted write blindly.

`pool_pre_ping=True` checks a connection when it is borrowed from the pool. If a database restart has made that pooled connection stale, SQLAlchemy can replace it. This helps explain the recorded local recovery check in which the same API remained running across a database stop and restart. It does not transparently replay a transaction that failed halfway through its work.

`hide_parameters=True` hides SQL parameter values in SQLAlchemy's relevant logging and error representations. It is useful defense against accidentally logging sensitive query arguments. It cannot sanitize every possible driver exception, custom log message or developer print statement. The health handler therefore also avoids logging raw database exception messages.

FastAPI gets a request session through this dependency:

```
def get_db(request: Request) -> Iterator[Session]:
    with request.app.state.session_factory() as session:
        yield session
```

The dependency creates a session, yields it to the endpoint, then closes it when the dependency finishes. It is scoped to one request and is not intentionally shared across simultaneous requests. Framework thread execution does not mean every user should share a session; mutable session state must never be used concurrently by unrelated work.

A SQLAlchemy session starts an implicit transaction when it first needs database work. The health endpoint only reads and never calls `commit()`. Closing the session ends that transaction and releases its connection, rolling back outstanding uncommitted transaction state. A rollback after a read is normal cleanup. Later business writes must commit deliberately; simply calling `session.execute` or returning HTTP 200 will not replace a correct commit.

`Base` inherits from `DeclarativeBase` and provides shared ORM metadata. Its Phase 1 body contains only a docstring because no business models exist yet. Metadata describes mapped application tables in Python. It is not the database itself, and it does not create tables merely because the class was imported. Alembic owns the explicit schema-change workflow.

Check your understanding

How many simultaneous checked-out connections can this engine allow? Up to ten under the configured pool and overflow limits; it does not eagerly open ten at startup.

Why does a closed read-only session roll back? Its implicit transaction still needs to end, even though no data was changed.

Can pre-ping safely repeat a half-completed future purchase or visit update? No. It repairs stale connection borrowing, not application transaction semantics.

7 The health endpoint

`backend/app/health.py` is the only implemented application endpoint. It answers a focused question: can this API execute a PostgreSQL query and a PostGIS query through its configured connection path? It does not claim that future business features exist.

The router's `@router.get` decorator registers `GET /health`. Its `response_model=HealthResponse` specifies the response structure, and its explicit 503 entry documents the dependency-failure response in OpenAPI. `tags=["health"]` groups the operation in Swagger. These are descriptions and framework instructions attached to the actual function below them.

The response class uses a small set of allowed text values:

```
class HealthResponse(BaseModel):
    status: Literal["ok", "error"]
    database: Literal["ok", "unavailable"]
    postgis: Literal["ok", "unavailable", "unknown"]
```

`Literal` restricts each field to the listed values. Pydantic models make that contract explicit and help FastAPI serialize it. There is no password, server address or detailed extension version in the public health body.

The session parameter is annotated with `Annotated[Session, Depends(get_db)]`. The first part describes the value as a session. The dependency marker tells FastAPI how to obtain it. You do not pass a session in an HTTP query string; FastAPI resolves the Python dependency while processing the request.

The first stage executes `SELECT 1`. This is a minimal real SQL statement. If a `SQLAlchemyError` occurs, the handler sets HTTP 503 and returns:

```
{"status":"error","database":"unavailable","postgis":"unknown"}
```

PostGIS is `unknown` because the handler never reached its check. The second stage runs `SELECT PostGIS_Full_Version()`. If that query raises a `SQLAlchemyError`, the response is:

```
{"status":"error","database":"ok","postgis":"unavailable"}
```

Here `database: ok` records that the first stage succeeded. It is not a permanent guarantee that the database cannot fail a moment later. The second failure can indicate missing PostGIS or another database error during that stage; the overall status remains unsuccessful. If both statements execute successfully, the handler returns HTTP 200:

```
{"status":"ok","database":"ok","postgis":"ok"}
```

The exception handling catches SQLAlchemy's error family. It does not catch every programming mistake and disguise it as a database outage. A bug unrelated to that error family should remain visible for investigation. For handled failures, the log records a safe message and the exception class, such as `OperationalError`, rather than the complete underlying exception text. This is why the tests inject fictional private details and verify that neither HTTP output nor captured logs contain them.

Readiness is a momentary dependency check. HTTP 200 does not prove the Alembic revision is current, all planned tables exist, a backup can be restored, or a customer can register. The endpoint does not inspect those things. Use the separate migration commands and the recorded test evidence for those conclusions.

When you receive 503, follow the failing stage. Verify container state, database logs and configuration, then issue SQL directly. When you receive 404, investigate the URL and route registration. When the connection is refused, investigate the listener and published port. Treat the HTTP status as evidence that narrows the investigation, not as an instruction to change random files.

Check your understanding

Why is PostGIS `unknown` after the first query fails? Its check was not attempted, so the server has no result to report.

Does a healthy response prove Phase 2 authentication works? No. Phase 2 has not been implemented, and `/health` tests only its two dependency statements.

Why log an exception class instead of the full exception? It preserves a useful error category while reducing the risk of exposing connection details or secrets.

8 Docker and the local network

The Python code does not run alone. It needs a Python interpreter, its installed packages, a PostgreSQL server, and a way to reach that server. Docker supplies a predictable environment for those pieces. Docker Compose describes how the two services start together on your computer.

An image a container and a volume

An image is a packaged filesystem and startup configuration. Think of it as a prepared runtime containing the tools and files a program needs. A container is a running instance created from that image. Stopping the container stops its process; rebuilding an image creates a new packaged version of the program.

Our API image is built from `backend/Dockerfile`. The database uses the existing `postgis/postgis:18-3.6` image. That image contains PostgreSQL and the spatial extension packages. There is no separate PostGIS server: PostGIS functions execute inside PostgreSQL.

A volume stores data independently of a particular container. The database writes its cluster into the named volume `mystreetvendor_postgres_data`. PostgreSQL 18 uses the mount target `/var/lib/postgresql` in this setup. The volume is why restarting or replacing the database container does not require rebuilding the database from nothing.

The distinction matters when debugging. Rebuilding the API image changes its packaged code. It does not change a database password already stored in the volume. Removing a stopped container is also different from removing the volume that holds your data.

Reading the Dockerfile

`FROM python:3.13-slim` selects the Python environment. `WORKDIR /app` makes `/app` the working directory for later instructions and for the running application. The `COPY` instructions place the package metadata, application, migrations, and tests inside the image.

The `ENV` settings also have specific jobs. `PYTHONDONTWRITEBYTECODE=1` avoids creating Python bytecode cache files, and `PYTHONUNBUFFERED=1` helps logs appear promptly. `PIP_DISABLE_PIP_VERSION_CHECK=1` skips pip's update announcements; it does not upgrade or loosen the pinned application dependencies.

The installation command uses `requirements.lock` as constraints and installs `.[test]`. The dot means the local Python package; the test extra adds the dependencies used by our automated tests. This is a development image, so keeping the test tools available is deliberate.

The Dockerfile creates `appuser`, grants that account access to the Pytest cache directory, and selects it with `USER appuser`. Installing packages during the build happens as root, but the finished API process runs as a non-root user. That fact was verified during the implementation milestone.

The final command starts Uvicorn with the application factory. `EXPOSE 8000` documents the intended container port. The Compose `ports` setting performs the actual publication to Windows. Editing a source file on Windows does not immediately edit the files already copied into a running image; rebuild and recreate the API after a code change.

Reading the Compose file

`infrastructure/compose.yaml` defines two services named `db` and `api`. The `db` service chooses its image and volume. The `api` service builds our Dockerfile. Compose places both on the project network and makes the service name `db` resolve to the database container.

These are the important addresses:

Where the caller runs	Destination	Address with default ports
Windows browser or PowerShell	Container API	`127.0.0.1:8000`
API container	Database container	`db:5432`
Optional Windows Python	Container database	`127.0.0.1:5432`

`127.0.0.1` is loopback: the current machine or network environment itself. Inside the API container, it refers to the API container, not the database. That is why Compose explicitly sets `POSTGRES_HOST=db` for the API.

Uvicorn listens on `0.0.0.0` inside its container so Docker can forward requests to it. Compose publishes that port on Windows loopback only. Those two settings describe different sides of the connection. The present setup does not provide a phone on the Wi-Fi network with an accessible API address.

Environment values have two jobs

Compose reads the root `.env` because our commands specify `--env-file .env`. Expressions such as `${POSTGRES_DB}` are replaced while Compose interprets the YAML. The explicit `environment` section then supplies values to each container. Reading an environment file for interpolation does not automatically copy every variable in that file into every container.

The `:?` form used for required values makes Compose fail early if a value is missing or empty. The database health command uses a doubled dollar sign, as in `$${POSTGRES_USER}`. This tells Compose to preserve the dollar expression so the shell inside the database container expands its own environment variable when the check runs.

The API container receives `POSTGRES_HOST=db` and port `5432`, regardless of the host connection values in `.env`. Windows Python reads the root `.env` through Pydantic Settings and uses the published port instead. Keep that distinction in mind when a connection works in one environment but fails in the other.

The database initialization values create a database and account when its volume is first initialized. Changing the password text in `.env` later does not issue an SQL password-change command against the existing cluster. Diagnose a credential mismatch before changing anything else.

The local bootstrap database account has privileges needed for initialization, migrations, and disposable test databases. It is a development account. The future production design separates migration privileges from the application account; that production configuration has not been built.

Health and restart settings

The database health check runs `pg_isready`. Compose waits for it before initially starting the API. The API health check calls `/health`, which performs the stronger PostgreSQL and PostGIS application queries.

Startup ordering does not apply migrations. It also does not guarantee the database will stay available. The API must handle later outages, which is why we tested a database stop and recovery. `restart: unless-stopped` concerns container restarts; an unhealthy label alone is not an automatic database repair procedure.

Check your understanding

1. Which object contains the program files: the image, the running process, or the named data volume?

2. Why would `POSTGRES_HOST=localhost` be wrong inside our API container?
3. Why can changing `.env` fail to fix a password mismatch?

Answers: the image contains the packaged program files; localhost points back to the API container; and the existing database volume retains credentials until an authorized database change updates them.

9 Schema changes with Alembic

A database schema describes the objects in a database: tables, columns, constraints, indexes, and extensions. Python source changes do not automatically update those objects. Alembic supplies an ordered history of database changes so each environment can be brought to a known revision.

The revision that exists today

The initial file is `backend/migrations/versions/0001_enable_postgis.py`. Its revision identifier is `0001_enable_postgis`, and `down_revision = None` means it has no earlier application revision.

Its upgrade contains one operation:

```
op.execute("CREATE EXTENSION IF NOT EXISTS postgis")
```

`IF NOT EXISTS` makes this step tolerate the image's already-enabled extension. More importantly, a clean database created without image initialization can still be prepared by the migration. Our integration tests deliberately use such a clean database.

Alembic separately stores the applied revision in `alembic_version`. That table is bookkeeping. Phase 1 does not create `users`, `vendor_profiles`, `visit_requests`, or the other business tables designed in Phase 0. Those designs will become migrations in the phases that actually use them.

The downgrade function intentionally does not remove PostGIS. The extension may have existed before this revision, and other spatial objects may depend on it. Moving the recorded application revision backward is therefore different from making the database indistinguishable from a server that never had PostGIS installed. Our downgrade tests run only in disposable databases.

How the migration environment connects

`backend/alembic.ini` identifies the migration directory and sets logging. Its `%(here)s` paths are based on the location of the configuration file. It contains no database password.

`backend/migrations/env.py` reads the same validated settings and creates the same kind of engine as the API. It can also accept a supplied connection. The tests use that option to direct migrations into their isolated databases without rewriting configuration files or putting credentials into Alembic's configuration text.

The environment passes `Base.metadata` to Alembic. Metadata is the Python description of application tables. It is currently empty because no business models exist. When future models are added, their modules must be imported so those table definitions are available to the migration environment.

Autogeneration compares the visible database schema with that metadata. It proposes changes; a developer must review the resulting migration. An automatically generated file is not proof that every proposed change is appropriate.

The PostGIS search path issue we found

The initialized image includes extension objects in schemas named `topology` and `tiger`. Its search path makes objects from those schemas visible. A search path controls where PostgreSQL looks when an SQL name is not schema-qualified.

The first live comparison saw geocoder tables and treated them as tables missing from our empty application metadata. It proposed removing them. No removals were executed by the comparison, but the result revealed a real migration boundary problem that the clean test database had not reproduced.

The fix has two parts. During the migration transaction, `SET LOCAL search_path TO public` restricts our application inspection to `public`. The `include_object` function also excludes the PostGIS-owned `spatial_ref_sys` table in that schema. The regression test places a table in `tiger`, makes it visible, and verifies that the comparison leaves it alone.

The three commands to distinguish

Read these examples first. Before executing them, complete the PowerShell session setup in Chapter 11 so that `@taskComposeArgs` is defined:

```
docker compose @taskComposeArgs run --rm api alembic current
docker compose @taskComposeArgs run --rm api alembic heads
docker compose @taskComposeArgs run --rm api alembic upgrade head
```

The container works in `/app`, which contains `alembic.ini`; that is why the shorter forms above find it. `current` reads the database's recorded revision. `heads` reads the latest revision in the migration files. `upgrade head` applies missing steps in order. After successful setup, the recorded revision should be `0001_enable_postgis` and match the head.

`alembic check` compares models and schema for changes that would need a new migration. `/health` answers a different question: whether the database and spatial function respond. A healthy API can still require a future schema migration. Learn to inspect both kinds of evidence.

Check your understanding

1. Does constructing a SQLAlchemy engine create tables?
2. Why does the initial downgrade leave PostGIS installed?
3. What is the difference between `current` and `heads`?

Answers: constructing an engine creates no tables; PostGIS may be shared or pre-existing; `current` reports the database's applied revision while `heads` reports the newest migration revision in the source files.

10 Tests and evidence

Testing asks a precise question about a behavior and checks the result. The foundation has ten automated cases. Seven test behavior without a real database; three use actual PostgreSQL and PostGIS. Both groups matter because they expose different mistakes.

Unit tests keep failure scenarios controllable

`test_config.py` contains four tests. They check URL construction with password punctuation, rejection of a missing password, rejection of an empty password, and parsing values from environment variables. They also check that normal settings and URL representations do not display the test password.

`test_health.py` contains three tests. They simulate healthy queries, a database failure, and a PostGIS failure. The failure tests check the HTTP status and response fields, and verify that private exception details do not appear in the response or captured logs.

The health unit fixture overrides FastAPI's database dependency with a mocked session. A mock is a controllable replacement: the test decides whether `execute` succeeds or raises an error. That makes a missing-extension scenario easy to reproduce. It does not demonstrate that Docker, networking, credentials, or a real database is working.

Integration tests cross the real boundary

The integration fixture creates a name such as `msv_test_` followed by a generated UUID. It validates the name before placing it in database creation or deletion SQL. Each test receives a separate database created from `template0`, rather than copying a development database with PostGIS already installed.

The first case proves the baseline enables PostGIS, reaches the expected revision, and tolerates a repeated upgrade. The second checks real health before and after migration, then checks geographical distance behavior. Two longitude/latitude points between 100 and 120 metres apart must be outside a 100-metre search radius and inside a 200-metre radius. This tests the use of metres rather than accidentally treating degrees as metres.

The third case checks downgrade and re-upgrade, preservation of PostGIS, and a clean schema comparison despite a visible table in `tiger`. Cleanup disposes connections and removes only the generated test database. It does not downgrade or delete your development database.

Opting into database work

The fixture requires `RUN_DB_TESTS=1`. Without it, the three integration cases are explicitly skipped. A skip means that case did not execute; it is not a passing database test. Complete the Chapter 11 session setup before running the Compose examples below.

```
docker compose @taskComposeArgs run --rm api python -m pytest
docker compose @taskComposeArgs run --rm -e RUN_DB_TESTS=1 api python -m pytest
```

Although the unit cases themselves need no database, these Compose commands inherit the API service's database dependency and wait for `db`. To run the unit cases without Docker, use the installed Windows virtual environment from the repository root:

```
.\.venv\Scripts\python.exe -m pytest -c backend/pyproject.toml backend/tests
```

Recorded evidence and your next run

The implementation verification recorded on 21 October 2026 reported the following final results:

Recorded run	Result	Duration
Windows with integration enabled	10 passed	2.79 seconds
Linux container with integration enabled	10 passed	2.23 seconds
Windows default run	7 passed and 3 skipped	0.68 seconds

Those are historical results from the foundation milestone, not new executions performed while writing this book. The same ten cases ran in both environments; that is not twenty different tests. Your run can take a different amount of time. Read its actual counts and failure output rather than expecting identical timing.

The recorded live checks also showed health changing from 200 to 503 during a database stop, then back to 200 after recovery without restarting the API. PostgreSQL reported version 18.6 and PostGIS 3.6.4. Image tags may receive patch updates, so a later rebuild can have different patch versions.

Passing these checks establishes a working foundation. It supplies no evidence that login, vendor discovery, requests, or push notifications work: their code and tests do not exist yet.

Check your understanding

1. What does a mocked database failure test prove?
2. Why create a database without PostGIS before testing its migration?
3. What does a report of three skipped integration cases tell you?

Answers: the mock proves our handler's response to the simulated failure; the clean database prevents image initialization from hiding a missing migration; skipped cases supply no fresh integration evidence.

11 Operating and debugging the foundation

The following exercises are for you to run. Their outputs are described as expected behavior, not as newly executed results. Work from the repository root and keep the actual output when investigating a failure.

Prepare one PowerShell session

```
Set-Location 'C:\Users\prana\Desktop\mystreetvendor'
$taskDockerBin = Join-Path $env:LOCALAPPDATA `
    'Programs\DockerDesktop\resources\bin'
if (Test-Path -LiteralPath $taskDockerBin) {
    $env:PATH = "$taskDockerBin;$env:PATH"
}
$taskComposeArgs = @('--env-file', '.env', '-f',
    'infrastructure/compose.yaml')
docker version
docker compose version
```

The backtick continues the PowerShell statement onto the next line. The argument array is expanded with `@taskComposeArgs`, called splatting. It saves retyping the environment-file and Compose-file options. Define these variables again in each new terminal; they are not permanent project settings.

Docker's per-user directory was missing from PATH during the original setup. Adding the directory made both `docker` and its credential helper discoverable. An all-users installation may use a different directory. Inspect the `Server` portion of `docker version`: the client executable existing is not proof the engine is available.

Your working `.env` already exists from setup. Do not replace it with the example file while following this book. On a genuinely new checkout, copy `.env.example` only if `.env` is absent and set a private password before starting. Do not print or share the secret file to diagnose a configuration problem.

Start services and inspect the first request

```
docker compose @taskComposeArgs config --quiet
docker compose @taskComposeArgs build api
docker compose @taskComposeArgs up -d --wait db
docker compose @taskComposeArgs run --rm api alembic upgrade head
docker compose @taskComposeArgs up -d --wait api
curl.exe -i --max-time 10 http://127.0.0.1:8000/health
```

`config --quiet` validates without displaying expanded configuration values. `--rm` removes the temporary migration container when the command ends. The migration runs explicitly between database startup and API startup.

Expected healthy output is HTTP 200 with `status`, `database`, and `postgis` equal to `ok`. The status line matters as well as the JSON. HTTP 503 means a dependency check failed. Connection refusal means no server accepted a connection at the address. Those are different failures and require different investigations.

Query the database directly

The database container already includes `psql`. Keep SQL on standard input to avoid nesting several layers of quotes:

```
$taskPsql = 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"'
@'
SELECT current_database(), current_user;
SELECT PostGIS_Lib_Version();
SELECT version_num FROM alembic_version;
'@ | docker compose @taskComposeArgs exec -T db sh -c $taskPsql
```

PowerShell passes this here-string into the Linux container. The `sh` command intentionally runs inside that container; the outer commands still use PowerShell. Expected evidence includes your configured database/user, a spatial version, and the initial revision. Extension tables can exist even though application business tables have not been created.

Find the failing layer

```
docker compose @taskComposeArgs ps
docker compose @taskComposeArgs logs --tail 100 api db
docker compose @taskComposeArgs run --rm api alembic current
docker compose @taskComposeArgs run --rm api alembic heads
```

Observation	First focused investigation
Docker command missing	Check this shell's PATH and the installation directory.
Engine unreachable	Check Docker Desktop, Linux context, and the server version response.

Observation	First focused investigation
Compose rejects a variable	Confirm the root directory and required values in the private environment file.
HTTP connection refused	Inspect container status and published port before editing database code.
Database authentication fails	Compare intended account configuration with the existing volume's credentials.
Host Python cannot resolve db	Use the loopback database address for the Windows process.
PostGIS unavailable	Confirm the database name, extension installation, and migration revision.
Edited code appears unchanged	Rebuild and recreate the API image because source is copied during build.

Record one symptom, choose one boundary, and repeat one check after a focused correction. A missing listener does not require rewriting the health handler. A missing migration does not require reinstalling Docker.

Observe a database outage and recover

Keep the API running during this exercise. The `finally` block attempts to restore the database even if the HTTP check encounters an error.

```
docker compose @taskComposeArgs stop db
try {
    curl.exe -i --max-time 10 http://127.0.0.1:8000/health
} finally {
    docker compose @taskComposeArgs up -d --wait db
}
curl.exe -i --max-time 10 http://127.0.0.1:8000/health
```

Expected behavior is 503 while the database is stopped and 200 after recovery. If recovery fails, inspect `db` status and logs and restore it before continuing. This exercise does not delete the volume. Do not run a development-database downgrade or add `--volumes` to repair a failed check.

When finished for the day, `docker compose @taskComposeArgs down` stops this project's stack while preserving its named database volume. Before the next phase, practise starting it again, inspecting the revision, running the full tests, and explaining which layer each command checks.

Check your understanding

1. Why does the command setup need to be repeated in a new terminal?
2. What would you inspect first after an HTTP connection refusal?
3. Why does the outage exercise use a `finally` block?

Answers: the variables and PATH change belong to one shell session; first inspect the listener/container and published port; the finally block attempts database restoration even if the intervening check fails.

12 Every file in the repository

The foundation checkpoint `699a820` contains 31 tracked files. This chapter is a map of that checkpoint. It tells you where to investigate a problem and where a future change belongs. You do not need to memorize the names; you should be able to find the right responsibility.

The four root files

`README.md` is the entry point. It identifies the milestone, gives the startup and test commands, and links to the longer explanations. Read it when returning to the project after a break.

`.env.example` is the public configuration template. Its password is a placeholder. The real `.env` is a private local file and is deliberately absent from the tracked inventory. Copying the template is a first-time setup action, not a reason to overwrite existing credentials.

`.gitignore` tells Git which local items to leave out, including `.env`, `.venv`, caches and backups. Ignoring a file does not encrypt it. It prevents an ordinary `git add` from including it in a new commit.

`.gitattributes` controls text-file handling, including line endings for the Python, YAML and documentation files. This matters because development happens on Windows while the application image runs Linux.

The application source

`backend/app/__init__.py` makes `app` a Python package. It contains a package description. Importing it does not load credentials or contact the database.

`backend/app/config.py` contains `Settings`. It reads and validates configuration, masks the password in normal representations, and builds a PostgreSQL URL without manually joining a password into a string. Investigate this file when environment values are missing or parsed incorrectly.

`backend/app/database.py` owns the connection engine and the per-request session dependency. `create_db_engine` configures pooling and timeouts. `get_db` yields a session and closes it after the request. `Base` is the starting metadata class for future models; it does not define business tables yet.

`backend/app/main.py` assembles the application. `create_app` creates the engine and session factory, registers the health router, and sets up shutdown cleanup. This is the file to read when asking how the small pieces become one running API.

`backend/app/health.py` defines the response model and `/health` route. Its two database operations check PostgreSQL and PostGIS. Its error branches return safe dependency statuses rather than raw connection errors. Start here when an HTTP response differs from the documented health contract.

Packaging and the development image

`backend/pyproject.toml` describes the Python package, supported Python version, direct dependencies, optional test dependencies and Pytest settings. Adding an approved Python dependency belongs here.

`backend/requirements.lock` records exact resolved dependency constraints. Installation uses it with `-c` together with the package declarations. Some dependencies are required on only one operating system. The successful Windows and Linux installations therefore do not imply identical lists of installed platform-specific packages.

`backend/Dockerfile` is the recipe for the development API image. It installs the package and tests, copies migration files and starts Uvicorn as `appuser`. A Dockerfile is a build recipe; editing it does not modify a container that is already running.

`backend/.dockerignore` excludes unnecessary local files from the backend build context. It reduces accidental inclusion of caches and environment files. The root `.env` is also outside that context because the context is the `backend` directory.

Migration files

`backend/alembic.ini` locates the migrations and configures their logging. Its paths use `%(here)s`, meaning the directory containing the configuration file. It contains no working database password.

`backend/migrations/env.py` connects Alembic to the same application settings. It supports normal execution, offline SQL generation and a supplied connection used by tests. It limits migration inspection to the application's schema and excludes the PostGIS-owned `spatial_ref_sys` table.

`backend/migrations/script.py.mako` is a template for future revision files. Alembic fills in its revision identifiers and upgrade/downgrade functions when a new migration is generated. It is not an HTTP handler or a migration that has already been applied.

`backend/migrations/versions/0001_enable_postgis.py` is the first actual revision. It enables PostGIS if needed. Its downgrade deliberately preserves the extension. Alembic separately tracks the revision in `alembic_version`; this file does not create user or vendor tables.

Test files

`backend/tests/conftest.py` supplies reusable fixtures. These include controlled unit settings, a mocked session, an application with its database dependency overridden, and a disposable database sandbox. Fixture setup and cleanup explain why tests can share preparation without sharing a development database's contents.

`backend/tests/test_config.py` has four cases: password punctuation in a connection URL, a missing password, an empty password and values read from environment variables.

`backend/tests/test_health.py` has three cases: a healthy response, a database failure and a PostGIS failure. The failure cases also check that private exception text does not appear in the HTTP response or captured logs.

`backend/tests/test_database.py` has three integration cases. They exercise clean and repeated upgrades, spatial distances and real health before/after PostGIS installation, and downgrade/re-upgrade with a schema-comparison regression check. These cases require `RUN_DB_TESTS=1` and use generated disposable databases.

Infrastructure and nine reference documents

`infrastructure/compose.yaml` describes the `db` and `api` services, their environment, health checks, local ports and persistent database volume. Investigate this file when container networking or service startup differs from your expectations.

The documentation has separate jobs:

- `docs/ARCHITECTURE.md` explains the full planned system, diagrams, module ownership and tradeoffs.
- `docs/DATABASE.md` describes the proposed business schema and geographical design, while identifying the much smaller current baseline.

- `docs/API.md` records the implemented health contract and marks future REST and WebSocket contracts as planned.
- `docs/SECURITY.md` describes authentication, authorization and privacy controls that must arrive with their features, alongside current development limits.
- `docs/DEPLOYMENT.md` explains local PowerShell workflows and future VPS and backup arrangements.
- `docs/DEBUGGING.md` helps identify which layer failed and records resolved setup issues.
- `docs/FILE_GUIDE.md` is the compact file reference and request walkthrough from which this longer study guide develops the teaching material.
- `docs/ROADMAP.md` defines phase boundaries, acceptance criteria and the pilot and scaling path.
- `docs/VERIFICATION.md` records the actual checks performed on 21 October 2026. Expected example output elsewhere is not a substitute for that evidence.

This handbook adds `docs/FOUNDATION_HANDBOOK.md` as the editable study text and `output/pdf/MyStreetVendor_Foundation_Handbook.pdf` as its reading copy. Neither adds application behavior. Generated local items such as `.venv`, caches and `.git` serve development or history. The Docker named volume stores the database and is not a repository file.

13 Practice and readiness for Phase 2

Reading gives you a model of the system. Practice checks whether that model helps you predict an outcome. The activities below are learner exercises; their expected results are not claims that they were rerun while writing this book.

Three practical tasks

Task one: trace a successful request. Use the setup from Chapter 11, confirm the stack is healthy, and request `/health`. Before opening any source file, write the order in which Uvicorn, FastAPI, the session, the driver and PostgreSQL participate. Then locate the two SQL statements and the response model in the code. Your explanation should include when the session is closed and why the connection may remain available in the pool.

Task two: compare migration evidence and test evidence. Inspect Alembic's current revision and head, then run the default test suite. Explain why the current milestone expects `0001_enable_postgis` and why the default test run can report three skips. Run the opt-in integration suite when the local database is available. Read one test assertion and explain what it proves. Do not downgrade the development database as a learning shortcut.

Task three: reproduce and recover from an outage. Follow the database-stop exercise in Chapter 11. Keep the API running, stop only this project's database, inspect the resulting health response, and restart the database. Confirm recovery before ending the exercise. Predict which health field should become `unknown` during the connection failure. Preserve the named volume throughout the exercise.

Ten questions before moving on

Answer these without looking at the next chapter, then compare your reasoning with the answer key.

1. `create_app` returned successfully. What must still happen before you can say PostgreSQL is reachable?
2. Why does the API container use `db:5432`, while Windows Python uses `127.0.0.1:5432` with the default configuration?

3. What do `SecretStr` and `.gitignore` protect, and what do they not protect?
4. PostgreSQL answers, but PostGIS is missing. What status and response fields should `/health` return?
5. What is the difference between Alembic's current revision and its head revision?
6. What should happen to database data after ordinary `docker compose down`, and where is that data stored?
7. Why are the three mocked health tests insufficient evidence that PostgreSQL/PostGIS works?
8. Why is one session per request useful, and does closing that session mean the entire engine is destroyed?
9. Why does the migration environment set its search path to `public` and ignore `spatial_ref_sys`?
10. Which important product capabilities are still absent, and what approval is needed before implementing the next phase?

You are ready for the next learning step when you can explain these answers, run the exercises, and investigate a failed check using evidence. This is learning readiness, not production readiness. Authentication, location privacy, notification delivery, public deployment and restore-tested backups each need their own implementation and verification.

Phase 2 will introduce authentication concepts such as password hashes, access tokens, refresh sessions and server-enforced permissions. This book does not implement them. Your separate approval remains necessary before that work begins.

14 Glossary and answer key

Words used in this foundation

API: the interface another program calls. Here, HTTP requests such as `GET /health` reach the backend through it.

ASGI: the Python server/application interface used by Uvicorn and FastAPI to exchange request and response events.

Uvicorn: the server process listening for HTTP traffic and invoking the application.

Framework: a library that supplies an application's structure and common behavior. FastAPI supplies routing, dependencies and response handling.

Driver: software speaking a database's network protocol. Psycopg is this project's PostgreSQL driver.

Engine: SQLAlchemy's database connection manager, configured with a URL, pool and connection options.

Pool: a limited collection of reusable database connections. A request borrows a connection instead of necessarily creating a new one each time.

Session: SQLAlchemy's per-operation workspace for database interaction and transaction handling. Our HTTP dependency creates one for a request.

Transaction: a unit of database work that can be committed or rolled back. Later business changes must define their complete transaction boundaries.

Metadata: SQLAlchemy's description of application tables and related schema objects. Current `Base.metadata` has no business models.

Extension: a PostgreSQL add-on installed within a database. PostGIS adds geographical types and functions.

Migration: a versioned database change applied deliberately. Alembic records which revision has been applied.

Image: the packaged recipe result from which a container starts. It contains the application's runtime and files.

Container: a running instance of an image with its own processes and network context.

Volume: storage managed separately from a container's replaceable filesystem. This project uses a named volume for PostgreSQL data.

Network: the connection environment through which services communicate. Compose supplies service-name resolution inside its project network.

Port: a numbered network endpoint. PostgreSQL listens on 5432 internally; the API listens on 8000.

Environment variable: a named configuration value supplied to a process. A `.env` file is one way tooling obtains such values; it is not automatically delivered to every container.

Mock: a controlled replacement used in a test. The mocked session lets a test simulate success or failure without a real database.

Integration test: a test exercising real components together. Here, these tests connect to PostgreSQL and install/use PostGIS in disposable databases.

Readiness: whether a service's checked dependencies can support its current work. This health route checks database and PostGIS access, not all future business features.

Answers to the ten questions

1. **Execute a real query.** Engine construction is lazy. Successful application construction does not establish a connection. A live health request or explicit SQL operation must obtain a connection and receive a result before reachability is demonstrated.
2. **The caller's network context differs.** `db` is the Compose service name resolved inside its network. Windows uses the published loopback port. Inside the API container, `localhost` means the API container itself, so it cannot be substituted for `db`.
3. **They reduce exposure in specific places.** `SecretStr` masks ordinary representations; it does not encrypt the value in memory. `.gitignore` excludes an untracked local file from normal staging; it does not encrypt the file, revoke a leaked secret, or remove previously committed history. The driver still needs the real password to connect.
4. **HTTP 503 with** `status: error`**,** `database: ok`**,** `postgis: unavailable`**.** The first query demonstrated database access; the second failed. A connection failure at the first stage instead reports `database: unavailable` and `postgis: unknown` because the spatial check was not reached.
5. **Current describes the database; head describes the revision files.** After a successful upgrade, the database should be at the expected head. A healthy spatial query alone does not prove the migration revision matches.
6. **Ordinary shutdown preserves the named volume.** Compose removes the containers and network while the PostgreSQL cluster remains in `mystreetvendor_postgres_data`. Removing volumes is a separate, destructive action and is not needed for this exercise.
7. **A mock tests behavior under supplied conditions.** It can prove the handler returns a safe 503 when an exception is simulated. It cannot prove DNS, credentials, the driver, the actual server or spatial functions work. The real integration cases supply that additional evidence.

8. **It separates concurrent work and ensures cleanup.** Requests should not share one mutable session. Closing a session releases its resources and ends its implicit read transaction; a borrowed connection can return to the engine's pool. The engine is disposed when the application shuts down.
9. **Extension objects are not application models.** The image exposes geocoder schemas through its search path. Alembic initially mistook their tables for removed application tables. Restricting inspection to `public` and excluding the PostGIS-owned table prevents that comparison error while keeping real application changes visible.
10. **The product features remain future work.** There is no implemented registration/login, vendor discovery, GPS tracking, visit workflow, realtime messaging, push delivery or mobile application. Phase 2 requires your approval and a scoped authentication implementation with its own tests. Understanding this foundation supports that decision without turning the current service into a finished product.